**Question:** Differentiate between the File System and Database System with respect to data redundancy, concurrency, and data integrity.

Ans:

- **Data Redundancy**:
  - *File System*: High redundancy, as the same data may be stored in multiple files.
  - *Database System*: Low redundancy due to normalization and centralized control.
- **Concurrency**:
  - *File System*: Poor concurrency control; multiple users accessing files may cause conflicts.
  - *Database System*: Strong concurrency control using transactions and locking mechanisms.
- **Data Integrity**:
  - *File System*: Integrity is hard to maintain; rules must be enforced by application programs.
  - *Database System*: Integrity is maintained through constraints (e.g., primary keys, foreign keys, checks).

**Question:** Discuss the roles of the Storage Manager and how it interacts with other components of a DBMS.

Ans:

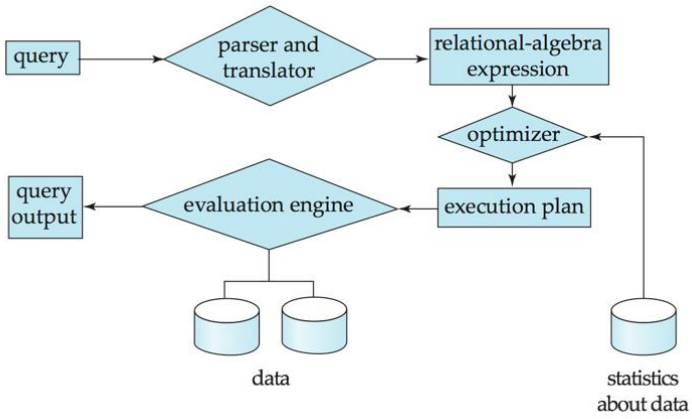
- **Role of Storage Manager**:
  The storage manager is a DBMS component that **stores, retrieves, and updates data** in the database. It ensures efficient access, consistency, and security of data stored on disk.
- **Main Functions**:
  1. **Authorization & Integrity Manager** – checks user access rights and enforces integrity constraints.
  2. **Transaction Manager** – ensures atomicity, consistency, isolation, and durability (ACID).
  3. **File Manager** – manages allocation of disk space and data structures.
  4. **Buffer Manager** – handles transfer of data between disk and main memory.
- **Interaction with Other Components**:
  The storage manager acts as a **bridge between the query processor and the physical database**. The query processor requests data → storage manager fetches it from disk/memory → results go back to the user.

**Question:** Explain Query processing system of DBMS with appropriate figure.

Ans:

**Query Processing System in DBMS**

- The **query processor** is responsible for **interpreting user queries** (usually in SQL), **optimizing** them, and **executing** them efficiently.
- Steps involved:
    1. **Parsing & Translation** – SQL query is checked for syntax/semantics and translated into an internal form.
    2. **Optimization** – The system chooses the most efficient execution plan (e.g., best join order, indexes).
    3. **Evaluation/Execution** – The optimized plan is executed, and results are retrieved from the storage manager.




**Question:** List and explain any four types of database systems. Highlight the key features and differences among them and indicate their application areas.

Ans:

**Four Types of Database Systems**

1. **Hierarchical Database**
    - *Structure*: Data is organized in a tree-like structure (parent–child).
    - *Key Feature*: Fast access for one-to-many relationships.
    - *Applications*: Banking, telecommunication, directory services.
2. **Network Database**
    - *Structure*: Uses a graph model with records (nodes) and links (edges).
    - *Key Feature*: Supports many-to-many relationships.
    - *Applications*: Inventory systems, supply chain, complex manufacturing.
3. **Relational Database**

- *Structure*: Data stored in tables (rows & columns), accessed using SQL.
- *Key Feature*: High flexibility, strong integrity, easy to use.
- *Applications*: Business applications, e-commerce, ERP, banking.

4. **Object-Oriented Database**
    - *Structure*: Stores data as objects (like in OOP).
    - *Key Feature*: Supports multimedia, complex data, and inheritance.
    - *Applications*: CAD/CAM, multimedia systems, scientific applications.

**Question:** Define Database Management System. Write about any two of the real-life applications of Database Management System.

Ans:

**Definition:**

A **Database Management System (DBMS)** is a software system that **allows users to define, create, maintain, and control access to databases**. It ensures efficient, secure, and consistent storage and retrieval of data.

**Two Real-Life Applications of DBMS:**

1. **Banking System**
    - DBMS stores customer accounts, transaction history, and loan information.
    - Allows fast transactions, account management, and fraud detection.
2. **Online Retail / E-Commerce**
    - DBMS manages product catalogs, customer data, orders, and inventory.
    - Supports real-time updates, recommendations, and secure payment processing.

**Question:** Describe about relational and object-oriented data model.

Ans:

**1. Relational Data Model**

- **Definition**: Organizes data into **tables (relations)** consisting of rows (tuples) and columns (attributes).
- **Key Features**:
    - Uses **SQL** for querying.
    - Ensures **data integrity** via primary keys, foreign keys, and constraints.
    - Supports easy data retrieval and manipulation.

- **Applications**: Business applications, banking systems, inventory management.

**2. Object-Oriented Data Model**

- **Definition**: Stores data as **objects**, similar to object-oriented programming, encapsulating both data and methods.
- **Key Features**:
    - Supports **complex data types** like multimedia, images, audio.
    - Allows **inheritance** and **reusability** of objects.
    - Useful for applications where relationships and behavior of data are important.
- **Applications**: CAD/CAM, multimedia systems, scientific research databases.

**Question:** Explain with example, how database management system resolve the problems of redundancy and atomicity.

Ans:

**1. Redundancy**

- **Problem**: Storing the same data multiple times leads to wastage of space and inconsistencies.
- **How DBMS Resolves It**:
    - DBMS uses a **centralized database** and **normalization** to eliminate duplicate data.
- **Example**:
    - Without DBMS (File System): Customer address is stored in both *Orders* and *Payments* files → duplication.
    - With DBMS: Customer data is stored **once** in a *Customer* table, referenced by Orders and Payments → redundancy removed.

**2. Atomicity**

- **Problem**: If a transaction is partially executed (e.g., debit succeeds but credit fails), the database becomes inconsistent.
- **How DBMS Resolves It**:
    - DBMS ensures **atomic transactions** using ACID properties. Either **all operations succeed or none**.
- **Example**:
    - Transferring $100 between two bank accounts:
        1. Debit from Account A
        2. Credit to Account B

- If step 2 fails, DBMS **rolls back** step 1 → atomicity maintained.

**Question:** Describe different types of DBMS users.

Ans:

**Types of DBMS Users**

1. **Database Administrator (DBA)**
   - **Role**: Responsible for **overall database management**, security, backup, and recovery.
   - **Tasks**: Defining database structure, granting user access, tuning performance.
2. **Database Designers**
   - **Role**: Design the **logical and physical structure** of the database.
   - **Tasks**: Creating schema, defining tables, relationships, and constraints.
3. **End Users**
   - **Role**: Use the database to perform daily tasks and retrieve information.
   - **Types**:
     - **Casual Users** – occasionally query the database.
     - **Naive/Parametric Users** – perform repetitive tasks using pre-defined queries.
     - **Sophisticated Users** – write complex queries and generate reports.
4. **Application Programmers**
   - **Role**: Develop programs and applications that interact with the database.
   - **Tasks**: Use APIs or embedded SQL to manipulate and retrieve data.

**Question:** Imagine you're developing a healthcare management system for a hospital. Different users like doctors, nurses, and administrative staff need varying levels of access to patient records. How can you provide users with an *abstract view of the data* in the database system? Explain with example.

Ans:

**Providing Abstract Views in a Database System**

- **Concept**: DBMS allows **views** to provide an **abstracted or simplified representation of data** for different users.
- **Purpose**: Users see only the data relevant to their role, **hiding sensitive or unnecessary information**.

**Example in Healthcare System**

1. **Doctors**

- Need full access to **medical history, lab results, prescriptions**.
- **View**: CREATE VIEW DoctorView AS SELECT PatientID, Name, MedicalHistory, LabResults FROM Patients;

2. **Nurses**
   - Need access to **current vitals and medications**, but not full history.
   - **View**: CREATE VIEW NurseView AS SELECT PatientID, Name, CurrentVitals, Medication FROM Patients;
3. **Administrative Staff**
   - Need access to **billing and contact info**, but not medical records.
   - **View**: CREATE VIEW AdminView AS SELECT PatientID, Name, Address, BillingInfo FROM Patients;

**Question:** What is data model? Briefly describe 2 different types of data model with examples.

Ans:

**Definition:**

A **data model** is a **conceptual framework** that defines how data is **structured, stored, and manipulated** in a database. It describes **relationships, constraints, and operations** on the data.

**Two Types of Data Models:**

1. **Relational Data Model**
   - **Structure**: Organizes data into **tables (relations)** with rows (tuples) and columns (attributes).
   - **Example**:
     - Table: Patients(PatientID, Name, Age, Diagnosis)
     - Each row represents a patient; columns store attributes.
   - **Use**: Banking, business applications, e-commerce.
2. **Object-Oriented Data Model**
   - **Structure**: Stores data as **objects** combining data and methods, similar to object-oriented programming.
   - **Example**:
     - Object: Patient { PatientID, Name, Age, MedicalHistory; displayHistory() }
     - Can handle complex data like images, videos, or multimedia reports.
   - **Use**: CAD/CAM, multimedia systems, scientific research.

**Question:** Consider a hotel reservation system for a chain of hotels. The system can handle room availability, guest reservations, and payment processing.
Issue-1: A guest successfully reserves a room but experiences a system failure prior to receiving a confirmation and before the room availability is properly updated.
Issue-2: Imagine two guests trying to book the same room at the same time.
Issue-3: A malicious user attempts to alter reservation records or manipulate room availability data.
How do the above issues relate to the problems faced by file systems while storing information? Explain with example how these can be resolved using database.

Ans:

**Problems in File Systems:**

1. **Issue-1: Partial Transactions / Atomicity Problem**
    - *File system problem*: If a crash occurs during a reservation, the system may record **incomplete data**, leading to inconsistencies.
    - *Example*: Guest reserves a room → system crashes before updating availability → room appears available to others, causing double booking.
2. **Issue-2: Concurrency Problem**
    - *File system problem*: Multiple users accessing the same file simultaneously may **overwrite each other's data**, leading to conflicts.
    - *Example*: Two guests book the same room at the same time → both records saved → double booking occurs.
3. **Issue-3: Security / Integrity Problem**
    - *File system problem*: File systems have **weak access control**, so malicious users can modify records.
    - *Example*: User manually edits a reservations file to change room availability.

## Resolution Using DBMS:

| Issue | DBMS Solution | Example |
|---|---|---|
| Partial Transactions | **Atomicity via transactions** | Use ACID transactions: either the reservation and availability update both succeed, or both roll back if failure occurs. |
| Concurrency | **Concurrency control & locking** | Two guests trying to book the same room: DBMS locks the room record during one transaction → prevents double booking. |
| Security & Integrity | **Access control & constraints** | Only authorized users can modify reservations; integrity constraints prevent invalid updates. |

**Question:** Define Database management system? What is data Abstraction? Write down the benefits of data abstraction.

Ans:

A **Database Management System (DBMS)** is a software system that **allows users to define, create, maintain, and control access to databases**, ensuring efficient, secure, and consistent storage and retrieval of data.

**Data Abstraction:**

- **Definition:** Data abstraction is the **process of hiding the details of how data is stored and maintained**, showing only relevant information to users.
- **Levels of Abstraction:**
    1. **Physical Level** – How data is stored on disk.
    2. **Logical Level** – What data is stored and relationships among data.
    3. **View Level** – What data is visible to a particular user.

**Benefits of Data Abstraction:**

1. **Simplicity:** Users see only relevant data without worrying about storage details.
2. **Security:** Sensitive data can be hidden from unauthorized users.
3. **Independence:** Changes in physical storage or structure do not affect user views.
4. **Consistency:** Multiple views can coexist without affecting the underlying database.

**Question:** How does database system ensures atomicity and concurrency? Explain each with example.

Ans:

**1. Atomicity**

- **Definition:** Atomicity ensures that a **transaction is treated as a single unit**: either all its operations are completed successfully, or none are applied.
- **How DBMS Ensures It:**
    - Uses **transaction management and rollback mechanisms**.
    - If any operation in the transaction fails, the DBMS **rolls back** all changes.
- **Example:**
    - Transferring $100 from Account A to Account B:
        1. Debit $100 from A
        2. Credit $100 to B
    - If step 2 fails, DBMS **undoes step 1**, keeping database consistent.

---

**2. Concurrency**

- **Definition:** Concurrency ensures that **multiple transactions can occur simultaneously** without causing data inconsistencies.
- **How DBMS Ensures It:**
    - Uses **locking, timestamping, or optimistic concurrency control** to manage simultaneous access.
- **Example:**
    - Two users try to book the same hotel room at the same time.
    - DBMS locks the room record during the first booking transaction → second transaction waits → prevents double booking.

**Question:** Compare between DML and DDL. Write down the functions of different component of storage manager.

Ans:

**1. Comparison between DML and DDL**

| Feature | DML (Data Manipulation Language) | DDL (Data Definition Language) |
|---|---|---|
| Purpose | Used to **manipulate data** in the database | Used to **define or modify database structure** |
| Examples | SELECT, INSERT, UPDATE, DELETE | CREATE, ALTER, DROP |
| Operation | Deals with **data stored in tables** | Deals with **schema, tables, indexes** |
| Execution | Affects **database content** | Affects **database structure** |

**2. Functions of Different Components of Storage Manager**

1. **File Manager**
    - Manages **disk space allocation** and organizes how data is stored in files.
2. **Buffer Manager**
    - Handles **data transfer between main memory and disk**, improving access speed.
3. **Transaction Manager**
    - Ensures **ACID properties** (Atomicity, Consistency, Isolation, Durability) for transactions.
4. **Authorization & Integrity Manager**
    - Enforces **user access rights** and **integrity constraints** (e.g., primary/foreign keys).

**Question:** How does a query being executed through different components of DBMS? Explain.

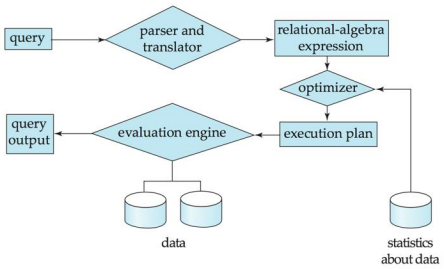
Ans:

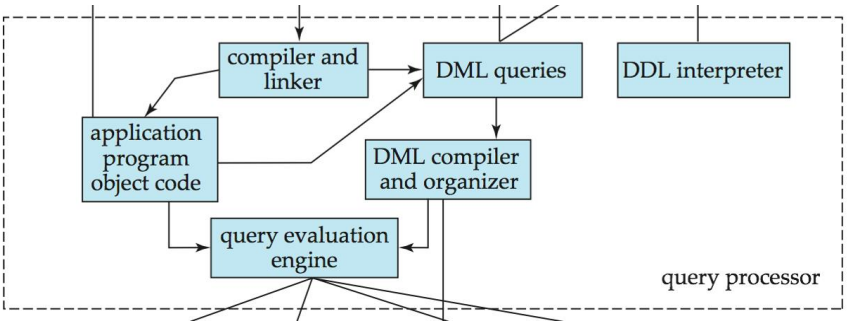
**Query Execution in DBMS**

When a user submits a query (e.g., SQL), it passes through several components of the DBMS before the result is returned.

**Steps and Components Involved:**

1. **Query Parser & Translator**
    - Checks **syntax and semantics** of the query.
    - Translates SQL into an **internal query representation**.
    - *Example*: SELECT * FROM Patients WHERE Age>50 → internal tree or relational algebra form.
2. **Query Optimizer**
    - Determines the **most efficient execution plan**.
    - Considers **indexes, join order, and access paths**.
3. **Query Evaluation / Execution Engine**
    - Executes the optimized plan.
    - Interacts with the **storage manager** to fetch, update, or insert data from disk.
4. **Storage Manager**
    - Handles **actual data retrieval** from disk or buffer.
    - Ensures **transactional consistency, concurrency control, and integrity**.
5. **Result Returned to User**
    - The evaluated results are formatted and presented to the user.






**Question:** Describe database instance and schema with example? What are the functions of a Storage Manager?

Ans:

**1. Database Schema and Instance**

- **Database Schema**
  - **Definition:** The **structure of the database**, describing how data is organized and the relationships among data.
  - **Example:**

    CREATE TABLE Patient (

    PatientID INT PRIMARY KEY,

    Name VARCHAR(50),

    Age INT,

    Diagnosis VARCHAR(100)

  - );
    - This defines the schema: table name, columns, and constraints.
- **Database Instance**
  - **Definition:** The **actual data stored in the database at a particular moment**.
  - **Example:**

| PatientID | Name | Age | Diagnosis |
|---|---|---|---|
| 101 | John Smith | 45 | Flu |
| 102 | Jane Doe | 30 | Diabetes |

    - This table represents the **current instance** of the database.

**2. Functions of Storage Manager**

1. **File Manager** – Manages **disk space and data structures** for storing files.

2. **Buffer Manager** – Handles **data transfer between main memory and disk** for faster access.
3. **Transaction Manager** – Ensures **ACID properties** (Atomicity, Consistency, Isolation, Durability).
4. **Authorization & Integrity Manager** – Enforces **access rights** and **integrity constraints** on data.

**Question:** Compare among Primary key, Super key, Candidate key, and Foreign key with examples.

Ans:

**Comparison of Keys in DBMS**

| Key Type | Definition | Example |
|---|---|---|
| **Primary Key** | A column or set of columns that **uniquely identifies each row** in a table. Only **one primary key** per table is allowed. | PatientID in Patient(PatientID, Name, Age) |
| **Super Key** | A set of columns that **uniquely identifies rows**, may contain extra attributes. | {PatientID, Name} or {PatientID} in the same table |
| **Candidate Key** | A **minimal super key** with no extra attributes; eligible to become primary key. | {PatientID} in the same table |
| **Foreign Key** | A column in one table that **refers to the primary key** of another table, used to maintain relationships. | DoctorID in Appointments(PatientID, DoctorID) referencing Doctor(DoctorID) |

**Question:** What is Data Model? Give examples of some popular data model. Compare between DDL and DML.

Ans:

**1. Data Model**

- **Definition:** A **data model** is a **conceptual framework** that defines how data is **structured, stored, and manipulated** in a database.
- **Purpose:** Describes **relationships, constraints, and operations** on the data.
- **Examples of Popular Data Models:**
    - **Relational Model** – Data stored in tables (e.g., MySQL, PostgreSQL).
    - **Hierarchical Model** – Data stored in a tree structure (e.g., IMS).
    - **Network Model** – Data stored in graph structures (e.g., IDMS).
    - **Object-Oriented Model** – Data stored as objects (e.g., OODBMS, PostgreSQL with JSON).

---

**2. Comparison between DDL and DML**

| **Feature** | **DDL (Data Definition Language)** | **DML (Data Manipulation Language)** |
|---|---|---|
| Purpose | Defines or modifies **database structure** | Manipulates **data stored in tables** |
| Examples | CREATE, ALTER, DROP | SELECT, INSERT, UPDATE, DELETE |
| Operation | Affects **schema** | Affects **database content** |
| Execution | Typically **auto-committed** | Can be **part of a transaction** |